

PixelShuffle#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.pixelshuffle.PixelShuffle.html>

Description:

Rearrange elements in a tensor according to an upscaling factor. Rearranges elements in a tensor of shape $(*, C \times r^2, H, W)$ to a tensor of shape $(*, C, H \times r, W \times r)$, where r is an upscale factor. This is useful for implementing efficient sub-pixel convolution with a stride of $1/r$. See the paper: Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network by Shi et al. (2016) for more details. `upscale_factor` (int) – factor to increase spatial resolution by

Function Signature

```
class  
torch.nn.modules.pixelshuffle.PixelShuffle(upscale_factor)  
[source]#  
  
Parameters  
  
Shape:  
  
extra_repr()[source]#  
  
Return type
```

Returns:

Tensor

Code Examples

```
>>> pixel_shuffle = nn.PixelShuffle(3)  
>>> input = torch.randn(1, 9, 4, 4)  
>>> output = pixel_shuffle(input)  
>>> print(output.size())  
torch.Size([1, 1, 12, 12])
```

Detailed Documentation

Documentation

Rearrange elements in a tensor according to an upscaling factor.

Rearranges elements in a tensor of shape $(*, C \times r^2, H, W)$ to a tensor of shape $(*, C, H \times r, W \times r)$, where r is an upscale factor.

This is useful for implementing efficient sub-pixel convolution with a stride of $1/r$.

See the paper: Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network by Shi et al. (2016) for more details.

`upscale_factor` (int) – factor to increase spatial resolution by

Input: $(*, C_{in}, H_{in}, W_{in})$, where $*$ is zero or more batch dimensions

Output: $(*, C_{out}, H_{out}, W_{out})$, where

Return the extra representation of the module.

Runs the forward pass.